

Aula 8 - Introdução ao Spring Boot: Controllers e Services

Duração: 4 horas (1h teórica + 3h prática)

Parte Teórica (1 hora)

1. Revisão de Java e HTTP (15 min)

- Conceitos-chave:
 - Protocolo HTTP (GET, POST, PUT, DELETE).
 - Arquitetura REST (Representational State Transfer).
 - Anotações Java (`@Annotation`).

2. Spring Boot Básico (25 min)

Arquitetura MVC do Spring

Diagram

Code

```
flowchart LR
    Client --> Controller
    Controller --> Service
    Service --> Repository
    Repository --> Database
    Client <--> Controller
    Controller <--> Service
    Service <--> Repository
    Repository <--> Database
```

Principais Anotações

Anotação	Camada	Descrição
@RestController	Controller	Define endpoints API REST
@Service	Service	Lógica de negócio
@Autowired	Injeção	Injeta dependências automaticamente
@RequestMapping	Controller	Mapeia URLs para métodos

Estrutura de Pastas

plaintext

```
src/main/java/  
└─ com.example.demo  
    └─ controller/ # Endpoints  
    └─ service/    # Regras de negócio  
    └─ repository/ # Acesso a dados  
    └─ model/      # Entidades
```

3. Ciclo de Requisição (20 min)

1. Cliente faz requisição HTTP.
2. `Controller` recebe e valida a requisição.
3. `Service` executa a lógica de negócio.
4. `Repository` acessa o banco de dados.
5. Resposta retornada ao cliente.

Parte Prática (3 horas)

Atividade: CRUD de Produtos com Spring Boot

Passo 1: Setup Inicial (30 min)

1. Crie um projeto no [Spring Initializr](#):
 - o Dependências: `Spring Web`, `Lombok`.
 - o Java 17.
2. Importe no IDE e crie as pastas básicas:

```
bash
```

```
mkdir -p src/main/java/com/example/demo/{controller,service,model}
```

Passo 2: Model e Controller (1h)

1. Classe `Product` (model):

```
java
```

```
@Getter @Setter @NoArgsConstructor @AllArgsConstructor  
public class Product {  
    private String id;  
    private String name;
```

```
        private double price;
    }
}
```

2. ProductController (controller):

java

```
@RestController
@RequestMapping("/products")
public class ProductController {

    @GetMapping
    public String hello() {
        return "API de Produtos!";
    }
}
```

3. Teste no navegador:

- o Acesse `http://localhost:8080/products`.

Passo 3: Implementar Service (1h)

1. ProductService :

java

```
@Service
public class ProductService {
    private List<Product> products = new ArrayList<>();

    public List<Product> getAll() {
        return products;
    }

    public Product create(Product product) {
        product.setId(UUID.randomUUID().toString());
        products.add(product);
        return product;
    }
}
```

2. Atualize o ProductController :

java

```
@RestController
@RequestMapping("/products")
public class ProductController {
    @Autowired
    private ProductService service;
```

```

    @GetMapping
    public List<Product> getAll() {
        return service.getAll();
    }

    @PostMapping
    public Product create(@RequestBody Product product) {
        return service.create(product);
    }
}

```

Passo 4: Testar com Postman (30 min)

1. Coleção de testes:

Método	URL	Body (RAW JSON)
GET	localhost:8080/products	-
POST	localhost:8080/products	{"name": "Notebook", "price": 3500.0}

2. Valide:

- POST → GET para ver o produto adicionado.

Tarefa para Casa

1. Implemente PUT e DELETE:

- Adicione métodos no `Service` e endpoints no `Controller`.

2. Validação básica:

- Use `@NotBlank` e `@Positive` nos campos do `Product`.

Próxima Aula

- Tópico: Spring Data JPA + MySQL.
- Atividade prática: Conectar API ao banco de dados.

Resultado Esperado

- API REST funcional com:
 - GET `/products` : Lista todos.
 - POST `/products` : Cria novo.
- Entendimento claro de:
 - Divisão entre Controller/Service.
 - Injeção de dependências.

plaintext

- ✓ Endpoints testados com Postman
- ✓ Código organizado em camadas
- ✓ Pronto para conectar ao banco de dados

<https://spring.io/images/spring-boot-flow.png> (*Fluxo Spring Boot*) 🚀